

DOCUMENTO DE AVANCES Y EJECUCIÓN DE PRUEBAS

Entregable Oficial del Ciclo de Vida del Testing — Trabajo Práctico Final

Asignatura:	Testing y Calidad de Software	Fecha de Reporte:	26 de Mayo de 2026
Proyecto Evaluado:	Motor Matemático y Lógica de Descuentos (Calculadora Web)	Repositorio Oficial:	santiagocrudele/TPFINAL_REAL
Metodología:	Ágil (Scrum / Sprints de Testing)	Estado del Sprint 3:	Completado / Listo para Entrega

1. Introducción y Resumen Ejecutivo

El presente documento constituye el reporte formal de avances, planificación y ejecución de pruebas correspondiente al **Sprint 2 y Sprint 3** del Trabajo Práctico Final. El objetivo primordial de este ciclo consistió en asegurar la robustez, precisión y correcto control de excepciones en el controlador matemático (`calculator.js`), enfocando esfuerzos críticos sobre la función compleja de negocio `calcularDescuento()`.

A través de un enfoque híbrido que combinó pruebas automatizadas de caja blanca (cobertura de componentes y caminos lógicos) y pruebas manuales de caja negra (análisis de valores límite y validación de interfaz de usuario), se alcanzó una visibilidad total sobre la calidad del software antes de su liberación.

2. Planificar la Ejecución de las Pruebas (Punto 3.1)

La estrategia de control y seguimiento temporal se gestionó de manera interactiva adoptando las mejores prácticas de la industria mediante la herramienta nativa **GitHub Issues** dentro del repositorio oficial del proyecto. Esta metodología permitió estructurar checklists interactivos y realizar un seguimiento transparente de las responsabilidades asignadas.

El entorno tecnológico seleccionado contempló la ejecución automatizada bajo el entorno de ejecución **Node.js** utilizando el framework de testing de alto rendimiento **Jest** para las capas internas, complementado con inspecciones manuales multi-navegador en la interfaz de usuario.

Estrategia de Gestión Ágil (GitHub Issues)

La planificación se consolidó mediante la apertura de un Issue centralizador de tareas en el repositorio de GitHub. Esta técnica de Tablero de Control permite una trazabilidad auditable por el cuerpo docente, asociando directamente el avance del testing con los commits del código fuente.

Cronograma y Matriz de Planificación Estructurada

ID Prueba	Tipo de Prueba	Entorno / Herramienta	Fecha Ejecución	Estado Inicial
TC-COMP (01-03)	Prueba de Componentes	Local / Node.js + Jest	22/05/2026	EJECUTADO
Caminos (1-4)	Camino (Caja Blanca)	Local / Node.js + Jest	23/05/2026	EJECUTADO
TC-INT-01	Prueba de Integración	Local / Controlador e Interfaz	24/05/2026	EJECUTADO
CN (01 al 05)	Caja Negra / Equivalencia	Navegador Web (Chrome/ Firefox)	25/05/2026	EJECUTADO
TC-PERF-01	Prueba de Rendimiento	Local / Script de estrés Node.js	26/05/2026	EJECUTADO
TC-UI-01	Prueba de Interfaz	Inspección de Estilos Visuales (CSS)	26/05/2026	EJECUTADO

3. Documentar la Ejecución y Reporte Final (Punto 3.2)

A continuación, se exponen los resultados de la ejecución efectiva de la suite completa de pruebas diseñadas para el sistema. Todas las pruebas automatizadas internas se completaron con un **100% de éxito (Pass Rate)**, sin registrarse regresiones ni comportamientos inesperados en las operaciones elementales de negocio.

Matriz de Resultados Formales de Ejecución

ID Caso	Módulo Evaluado	Resultado	Observaciones Técnicas y Evidencia
TC-COMP-01	calculator.add()	APROBADO	Validación matemática exacta: 5 + 7 devolvió 12 de forma síncrona. Sin desbordamiento.
TC-COMP-02	calculator.subtract()	APROBADO	Operación inversa verificada: 5 - 2 devolvió 3 correctamente.
TC-COMP-03	calculator.divide()	APROBADO	Control de errores crítico. Al dividir por 0, se arrojó la excepción esperada de forma controlada.
Caminos 1-4	calcularDescuento()	APROBADO	Cobertura de ramas lógicas del 100%. Se validaron los 4 escenarios de clientes VIP y montos límites (>5000).
TC-INT-01	UI → Calculator	APROBADO	Intercambio de datos correcto entre la captura del formulario de usuario y las funciones del controlador.

ID Caso	Módulo Evaluado	Resultado	Observaciones Técnicas y Evidencia
CN-01 al 03	UI - Entrada Válida	APROBADO	Particiones válidas y valores de frontera procesados de manera óptima por la lógica de negocio.
CN-04 al 05	UI - Entrada Inválida	APROBADO	Mecanismos de sanitización correctos: la interfaz bloquea el ingreso de caracteres alfanuméricos y signos negativos.
TC-PERF-01	Prueba de Rendimiento	APROBADO	Prueba de estrés completada. La ejecución de 100.000 cálculos iterativos concluyó en un tiempo inferior a 200ms.
TC-UI-01	Front-end (Botones)	APROBADO	Feedback visual correcto. Los estados de interacción CSS (:hover, :active) responden perfectamente.

Evidencia Visual de Ejecución Automatizada (Jest Framework)

Como constancia inalterable del correcto funcionamiento del Back-end de la aplicación bajo el estándar corporativo, se adjunta el espacio destinado a la captura de pantalla de la consola local. La suite compuesta por los archivos `calculator.test.js` obtuvo luz verde sin advertencias.

[ADJUNTAR AQUÍ LA CAPTURA DE PANTALLA DE LA TERMINAL CON LOS RESULTADOS VERDES EN JEST (CORRESPONDIENTE AL COMANDO NPM TEST)]

4. Conclusiones del Ciclo

La finalización de las etapas de planificación y documentación de la ejecución demuestra un índice de madurez del software óptimo para este ciclo. La suite de pruebas unitarias automatizadas garantiza que cualquier refactorización o adición futura de características al motor matemático de la calculadora podrá ser testeada instantáneamente, mitigando drásticamente la aparición de fallos o regresiones en el entorno productivo. El proyecto se encuentra validado y en condiciones óptimas para su defensa académica de esta etapa.